

יסודות הבדיקות (CTFL פרק 1)

פרק A: מהי בדיקה

- **Test Objectives (מטרות הבדיקה):** בדיקה משרתת מטרות שונות: למשל וידוא שהמוצר עומד בדרישות, איתור פגמים, בניית ביטחון באיכות והפחתת סיכונים [L245-L253†23] [L121-L124†42]. בדיקות לא נועדו להוכיח שאין באגים, אלא להקטין את הסיכוי שעוד קוד שגוי נותר במערכת [L121-L124†42]. רמת K1: (זיהוי מטרות טיפוסיות). **זווית מבחן:** לעיתים ישאלו מיהם יעדי בדיקה סטנדרטיים (כמו "בניית אמון באיכות", "ודאות בדרישות", "מציאת באגים" וכו') [L245-L253†23]. **טעות נפוצה:** לחשוב שאם לא נמצאו באגים – התוכנה נקייה באופן וודאי (מטלית רעלי הפרדוקס של היעדר שגיאות) [L121-L124†42] [L20†L64-L68].
- **Testing vs Debugging (בדיקה לעומת ניפוי שגיאות):** בדיקה היא פעילות לצורך מציאת כשלים או איסוף מידע על איכות התוכנה, בעוד ש־debugging הוא תהליך מציאת ופתרון הגורם לשגיאה שהתגלתה [L273†23]-L281. למשל, בדיקה דינמית עשויה להפעיל את התוכנה כדי לגרום לכשל, ואילו ניפוי השגיאות יתמקד באיתור ושכתוב הקוד שמפיק את השגיאה [L273-L281†23]. רמת K2: (הבחנה בין הפעילויות). **זווית מבחן:** שאלה טיפוסית לגיד שמתבצעת פעולת **בדיקה** ואז נדרשת פעולת **debugging** למציאת שורש התקלה [L273†23]-L281. **טעות נפוצה:** להתבלבל ולהבין שבדיקה בעצמה מתקנת בעיות בקוד – אלא רק מדווחת עליהן.
- **What testing can/cannot prove (מה בדיקות יכולות להוכיח ומה לא):** בדיקה יכולה להוכיח שרקמות (units) מוגדרות מתפקדות כנדרש, אך לא יכולה להוכיח שהתוכנה ללא שגיאות כלל [L121-L124†42] [L64-L68†20]. כלומר, אף שאנו עשויים למצוא באגים רבים, הבדיקות אינן מבטיחות אפס באגים. רמת K2: **זווית מבחן:** מפרץ כללי בבחינה על עיקרון הבדיקות – קביעה שטעונה זיהוי מתודה או מונח שגוי כמו "בדיקה היא הוכחת העדר שגיאות" היא שגויה. **טעות נפוצה:** להסיק שבדיקות מלאות ווידאו אותן יבטיחו תוכנה מושלמת.
- **Testing vs QA vs QC (בדיקה מול הבטחת איכות QC ובקרת איכות QA):** בדיקות (Testing) הן פעולה לבחינת המערכת וגלוי פגמים, כחלק מתהליך בקרת איכות (QC). **אבטחת איכות (QA)** היא גישה ניהולית-פרוצדורלית לשיפור תהליכי הפיתוח ולמניעת כשלים מראש [L46-L54†27] [L12†L53-L57]. QA עוסק בקביעת תקנים ושיפור התהליך, QC עוסק בבדיקה השיטתית של התוצר הסופי, ובדיקות הן כלי עיקרי בתוך הגדרות ויש לשייך למי מהן QA, QC או בדיקות [L46-L54†27] [L27†L46-L54]. רמת K2: **זווית מבחן:** שאלות על ההבדל בין המונחים – למשל יתנו תוכנה הוא למעשה משקיף או "QA" (כמו שמהדר עצמו לאט את איכות הקוד), במקום להבין שבעצם הוא חלק מבקרת האיכות ואחראי לאיתור כשלים במערכת.

פרק B: שבעת עקרונות הבדיקה

על פי הסילבוס, לכל אחד משבעת העקרונות תיאור ותוצאה במקרה של הפרה:

- **Presence of defects (קביעת נוכחות פגמים):** בדיקות חשופות קיומם של פגמים אך אינן מבטיחות העדרם [L81-L85†31] [L121-L124†42]. כלומר, אמנם גילוי באג בודק שהמערכת אינה נקייה, אך אי גילוי אינו אומר שהיא תקינה לחלוטין. דוגמה: בדיקות אינטגרציה מצביעות על תקלות, אך גם אם כל הבדיקות עברו בהצלחה עדיין לא בטוח שאין באגים. הפרה: אם מתכנת יאמר "לא מצאתי באגים, לכן אין" זו תפיסה שגויה [L59-L67†20].
- **Exhaustive testing is impossible (אין בדיקה ממצה):** אי אפשר לבדוק את כל השילובים האפשריים של כניסות ומצבים במוצר (חוץ מתוכנות טריוויאליות). לכן משתמשים בסלקציה חכמה בהתאם לסיכון ולחשיבות חלקי המערכת [L87-L92†31] [L20†L83-L90]. הפרה: ניסיון לבדוק הכל בלי התחשבות בסיכון יגרום לבזבוז זמן רב ויאלץ לדלג על אזורים חשובים. ברוב הבחינות משנים זו שאלה "מדוע אנו משתמשים בבדיקות מבוססות סיכונים?" כדי לבחון הבנה שנדרש להתמקד במקומות הרגישים.
- **Early testing (בדיקה מוקדמת):** מומלץ להתחיל בבדיקות בכל שלב מוקדם של התהליך (למשל איתור פגמים כבר בדרישות או בעיצוב) [L93-L96†31] [L20†L76-L83]. דוגמה: קיום ביקורת על דרישות טרם הקידוד כדי

- לגלות אי-בהירות. הפרה: אם מתחילים לבדוק רק אחרי הפיתוח (לדוגמה, מבצעים בדיקות רק בשלב האחרון), הפגם שמתגלה עשוי לדרוש תיקונים נרחבים ובהוצאות גדולות יותר [L76-L83†20].
- **Defect clustering (צבירת פגמים):** בפועל, רוב הבאגים מצטברים בכמה מרכיבים מרכזיים במערכת (עקרון הפרנדה 80/20) [L98-L101†31]. דוגמה: אחר בירור חוזר של מודול אחד נמצא שרוב הקריסות נגרמות שם, ולכן מקצים לו יותר בדיקות רגרסיה. הפרה: חלוקה שווה של מאמץ בכל מערכת (ללא זיהוי אזורים צפויים לתקלות) עלולה לפספס נקודות תקלה קריטיות.
 - **Pesticide Paradox (פרדוקס קוטל החרקים):** אם מריצים שוב ושוב את אותם מקרי בדיקה, בסופו של דבר הם יחדלו למצוא פגמים חדשים [L105-L111†31]. כדי למנוע זאת, יש לעדכן ולשנות מקרים קיימים, להוסיף תרחישים חדשים ולגוון טכניקות בדיקה. דוגמה: ריצה אוטומטית יומית של אותו סט בדיקות ללא עדכון תניב "כיסוי" גבוה למכונה אך תחמיץ באגים חדשים. הפרה: הימנעות מרענון מערך בדיקות תוביל לכך שהמערכת תתקדם והשינויים בה לא יתגלו בבדיקות הישנות.
 - **Context-dependent testing (בדיקות תלויות הקשר):** יש להתאים את שיטת הבדיקה והכלים לפי סוג הפרויקט והקונטקסט. למשל, בדיקות תוכנות בטיחות קריטיות יתמקדו בדרישות אבטחה ואמינות (כמותן הרבה יותר קפדניות), לעומת בדיקה של אפליקציה רגילה [L113-L116†31]. הפרה: שימוש באותן שיטות בדיקה לכל פרויקט מבלי להתחשב ביעוד או בסביבת הפיתוח עלול להביא לבחירת שגויה של כלי בדיקה או קידוד בלתי הולם.
 - **Absence-of-errors fallacy (אשליית העדר שגיאות):** אין ערובה שאפליקציה נטולת באגים תענה על צרכי הלקוח [L64-L68†20] [31†L118-L121]. דוגמה: תוכנה עשויה להיות נטולת באגים פונקציונליים אך כושלת כיוון שהיא לא עונה על דרישות המשתמש (ייתכן שהוגדרו דרישות שגויות מלכתחילה). הפרה: הסתמכות רק על איתור ותיקון באגים (לדוגמה, על ידי המדד "מספר הבאגים שתוקנו") בלי לבדוק התאמה לדרישות העסקיות תוביל לכך שתוצר לא יענה על הצרכים האמיתיים [L64-L68†20].

פרק C: שגיאה-פגם-כשל ועוד

- **Error / Mistake (שגיאה אנושית):** טעות חשיבה או פעולה שגויה של אדם (למשל המתכנת, האנליסט או הסקריפטור) שיוצרת קוד שגוי. שגיאה לא מתגלת מיידית (אם תוקנה בהרצה או ע"י קריאה) לפני שהופכת לפגם. רמת K1: **זווית מבחן:** לעיתים שואלים להבחין בין "mistake" כהגדרה אנושית לבין "defect" בקוד [L146†42]. [L154]. **טעות נפוצה:** לבלבל שגיאה (mistake) עם פגם בקוד; בפועל שגיאה היא ההתנהגות של המתכנת ולא המערכת.
- **Defect/Bug (פגם/באג):** לקוי או "באג" בקוד המערכת או בתוצרי פיתוח אחרים שבו המערכת לא עומדת בדרישות. הפגם הוא תוצר השגיאה האנושית ומונע מהתוכנה לעבוד כנדרש [L201-L207†34]. רמת K1: **זווית מבחן:** שנתנו קוד שמפגין ביצועים לא נכונים ושואלים האם מדובר בשגיאה של המפתח, בבאג/פגם בקוד או בכישלון בבדיקת משתמש [L201-L207†34]. **טעות נפוצה:** לחשוב שבאג הוא תמיד פעולה של מתכנת בלבד; גם מסמך דרישות יכול להכיל "באג" (פגם בתכונות הדרושות).
- **Failure (כשל):** התנהגות שגויה של התוכנה בריצה (בזמן ביצוע), כשפונקציה נכשלת או שלא עומדת במה שציפנו [L268-L270†34]. הכשל נובע מהפעלת פגם. רמת K1: **זווית מבחן:** בדרך כלל בוחנים אם יודעים לזהות תרחיש "כישלון" על פי התיאור – למשל, "התוכנה קיבלה קלט חוקי אך לא החזירה תוצאה צפויה" הוא כשל. **טעות נפוצה:** להתבלבל ולקרוא "כישלון" לרישום תקלה (לדוגמה במערכת ניפוי באגים כשפותרים את הפגם) במקום לתיאור התנהגות שגויה בתוכנה בפועל.
- **Root Cause (גורם שורש):** הסיבה העיקרית להיווצרות פגם או כשל – לרוב שגיאה בתהליך קודם (דרישות, עיצוב או קידוד). למשל, אם תקלה נובעת ממתכון אלגוריתם שגוי, גזירת המסקנה הלא נכונה היא שורש הבעיה. רמת K2: **זווית מבחן:** ייתכן ששואלים לבחון במה מטפל ניתוח הסיבות (לרוב בשאלות על ניהול פגמים). **טעות נפוצה:** להסתפק בתיקון המיידי של הפגם בלי לבדוק לעומק היכן נעשתה השגיאה שגרמה לו, ואז לפגוש באג דומה במקום אחר.
- **False positive/negative (חיובי/שלילי שגוי):** תוצאה שגויה של בדיקה: **חיובי שגוי** (False Positive) – הבדיקה דיווחה על באג כאשר למעשה אין בגד בקוד [L97-L105†37]. למשל, בדיקה אוטומטית שמציינת "באג" מכיון שהממשק בלתי פעיל מסיבות חיצוניות. **שלילי שגוי** (False Negative) – הבדיקה נכשלה לאתר באג קיים [L125-L133†37]. למשל, טעות שתוקנה בקוד אך המבחנים לא תפסו אותה והשאירו פגם רציני מבלי להצביע עליו. רמת K2: **זווית מבחן:** שאלה על הגדרה ותוצאה של FP/FN, או זיהוי FP/FN מתיאור מצב. **טעות נפוצה:** להניח שתוצאות בדיקות אוטומטיות הן תמיד מדויקות; בפועל, FP גורמים לפספוס בעיות אמיתיות ויצירת

“מצוקה אלבומי צעקה” (cry wolf) [37†L97-L105], ו־FN מנבאים עלולים לגרום לתוכנה יציבה להתגלגל עם באגים.

- **Failure without defect, defect without failure (כשל ללא פגם ולפחך):** יתכן מצב בו מתרחש כשל ללא פגם בקוד (למשל כשנפלה תקלה בסביבת ההרצה – כשל שנגרם כתוצאה מבעיות חיצוניות) [L158†42]. ולהיפך, פגם יכול להיות במערכת אך לא להזעיק כשל כי נתיב הקוד אינו מבוצע בשום מבחן [L158†42].
- [L161]. הסבר: לפגם שהקוד לא הוצא לפועל (כמו קוד מת שהתוכנה לא משתמשת בו), לא תהיה תוצאה בלתי צפויה בזמן ריצה, ולכן אין כשל נצפה. רמת K2: **זווית מבחן:** לעיתים יתנו תרחיש שבו הכשל נוצר מסיבה חיצונית ויבקשו להסביר שאין פה פגם בקוד – לדוגמה תקלת חומרה או התקנת תוכנה לא נכונה. **טעות נפוצה:** להאמין שכל כשל הוא בהכרח קוד שגוי; לעיתים “כשלים” נובעים מסיבות סביבתיות (אל-תוכנתיות), ולהפך.

פרק D: תהליך הבדיקה

- **תכנון הבדיקות (Test Planning):** פעילות בה מגדירים את אסטרטגיית הבדיקה, היקף הפיצ'רים שנבדקים, לוחות זמנים, משאבים וקריטריוני הצלחה [L70-L78†48]. התוצר העיקרי הוא מסמך תוכנית בדיקות (Test Plan) המתאר מטרות, היקף, טכניקות, תזמון ודרישות (כגון נתוני בדיקה, סביבת בדיקה) [L70-L78†48]. רמת K2: **זווית מבחן:** לרוב שואלים מהו האחראי על תכנון הבדיקות ומה נכלל בתוכנית (ובדיקות נכונות של מרכיבי ה־Test Plan). **טעות נפוצה:** לצפות שהתוכנית קבועה – בפועל היא יכולה להתעדכן כשמתגלים סיכונים חדשים או תלויים חיצוניים (משתמע מהקשרים).
- **ניטור ובקרה (Test Monitoring & Control):** עוקבים אחר התקדמות הבדיקות ובוחנים האם התהליך מתנהל לפי התוכנית (נתונים כמו כמות מקרי בדיקה מקודמים/בוצעו, באגים שהתגלו, מדדי זמן). אם יש חריגה (לדוגמה איחורים או ריבוי באגים), מבצעים פעולות תיקון (כגון הקצאת משאבים נוספים) [L87-L93†48]. התוצרים: דוחות סטטוס והמלצות להמשך (Test Progress Reports). רמת K2: **זווית מבחן:** שאלות על איך מתריעים על בעיות זמנים/איכות בפריקט הבדיקות. **טעות נפוצה:** להניח ש”אם הכל מתנהל לפי התוכנית אין צורך לעקוב” – בפועל צריך מעקב מתמיד כדי לזהות מוקדם בעיות (כתלות בסילבוס ודרישות הפרויקט).
- **ניתוח הבדיקה (Test Analysis):** שלב שבו בוחנים את הדרישות, התכולה והטקסטים הטכניים כדי להפיק מהם אילו תרחישי בדיקה דרושים (test conditions) [L48†L98-L105]. עובדים על פירוק הדרישות לאלמנטים הניתנים לבדיקה (למשל use cases או business rules). התוצרים העיקריים הם דרישות בדיקה (test basis) ותנאי בדיקה (test conditions). רמת K2: **זווית מבחן:** שאלה על מה נכלל בשלבי ניתוח (למשל זיהוי תנאי בדיקה ומתן דוגמאות). **טעות נפוצה:** לדלג על ניתוח וליצור מקרים ישירות – סיכון להחמיץ תרחישים מכסה דרישות לא מוגדרות.
- **עיצוב הבדיקה (Test Design):** בשלב זה בונים את מקרי הבדיקה ואת התסריטים המפרטים איך להפעיל את הבדיקות. עיצוב הבדיקה עונה על השאלה “כיצד נבדוק” – הגדרת ערכי קלט, פעולות, תוצאה צפויה ועוד [L113-L120†48]. תוצרים: ערכת תרחישי בדיקה, פרוצדורות בדיקה ורשימת תנאי קלט. בדרך זו גם מקשרים באופן דו-כיווני בין הדרישות למקרי הבדיקה (מטריצת traceability) [L48†L113-L120]. רמת K2: **זווית מבחן:** יכול לשאול איך בנוי תרחיש בדיקה, או לדוגמה להזכיר מטריצת traceability כפריט עיצוב בדיקות [L113†48].
- [L120]. **טעות נפוצה:** להתחשב רק בתסריטים חיוביים (positive) ולדלג על תרחישי שגיאה; בנוסף, לא לבחון כראוי הנחות סביבה ודאטה דרושים.
- **מימוש הבדיקות (Test Implementation):** בשלב זה מפתחים ומכינים את הכלים והתשתית להפעלת הבדיקות. פעילויות כוללות יצירת הסקריפטים האוטומטיים (אם יש), הכנת סביבת הבדיקה (למשל הדמיות, התקנת תוכנות נלוות) והטענת נתוני הבדיקה [L122-L131†48]. התוצרים: סטים של רכיבי בדיקה ובדיקות אוטומציה מוכנות להרצה, כמו גם תיעוד סביבת הבדיקה. רמת K2: **זווית מבחן:** מה מבדל שלב מימוש משלבים אחרים (למשל הוצאה לפועל וכו'). **טעות נפוצה:** להתעכב במימוש רב מדי של כלי אוטומציה בשלב מוקדם מדי (לעיתים אין צורך בבניית אוטומציה מלאה ביחידות), במקום להתמקד בהגדרת מקרי בדיקה ידניים ראשוניים.
- **ביצוע הבדיקות (Test Execution):** הפעלת מקרי הבדיקה על המוצר בפועל. מתעדים איזה גרסה של המוצר וכלי הבדיקה בשימוש, מפעילים את הבדיקות בעזרת כלים ידניים או אוטומטיים, ומשווים תוצאות בפועל לצפויות [L139-L146†48]. בפועל: מדווחים על כשלים (שמטפלים בהם על ידי המפתחים), ובוחנים את אמינות התיקונים (בדיקות רגרסיה). התוצרים: לוג בדיקות, רשומות כשלונות ודו"ח כיסוי בדיקות ראשוני. רמת K2: **זווית מבחן:** לרוב שואלים לדוגמה איך נרשם כשל (log) ומה עושה ה־test executor. **טעות נפוצה:** להזניח רישום מסודר של נתונים (לוגים ותוצאות) – מה שעלול להקשות על חזרה על הבדיקה או על ניתוח תקלות במועד מאוחר יותר.

- **סיום הבדיקות (Test Closure):** השלב בו סוגרים רשמית את פעילות הבדיקה בפרויקט או באיטרציה. בודקים שהשגת היעדים (לדוגמה, נסגרו כל באגים קריטיים, עמדו בקריטריונים ליציאה) [L148-L154+48], מסכמים את תוצאות הבדיקות ודוחקים על קבלת מוצר או המשך הפיתוח. התוצאה העיקרית היא דוח סיכום בדיקות המסכם אילו פגמים נמצאו, מדדי כיסוי והמלצות (Test Summary Report) [L148-L154+48]. רמת K2: **זווית מבחן:** שאלת זיהוי פעילויות סיום – למשל שאלה "מה כולל דוח סיכום בדיקות?". **טעות נפוצה:** לבצע "סגירת בדיקות" רק באופן פורמלי בלי לממש משוב – כך שהלימוד מלקחים יימנע והתקלות יחזרו בפרויקטים עתידיים.
- **Traceability (מעקב/Traceability):** מעקב אחר דרישות מפתח לבדיקות המתאימות בהן. מטריצת traceability מקשרת כל דרישה למקרי הבדיקה שתוכננו עבורה [L119-L127+45] [L113-L120+48]. בכך ניתן להראות שאף דרישה לא נשכחה וכמה בדיקות כיסו כל אחד מהתנאים. רמת K2: **זווית מבחן:** לעיתים יובא partial scenario ויש לשאול מה יחס הדרישות למבני בדיקה. **טעות נפוצה:** לוותר על מסמך traceability – ואז עלול לצאת שדרישה חשובה לא נבדקה כלל.
- **השפעת הקשר (Context):** תהליך הבדיקה מושפע ממדיניות הפרויקט, מודל הפיתוח ומאפייני המוצר. לדוגמה, בפרויקט גמיש (Agile) יהיו איטרציות קצרות ותיקשורות רציפה עם הלקוח, ובבדיקות יתמקדו באוטומציה נרחבת; בפרויקט מונחה מסמכים רב (דוגמת מתודולוגיית V) יהיו שלבים ברורים של תכנון ובקרה מוגדרת. רמת K2: **זווית מבחן:** לעיתים שואלים כיצד מתהליך הבדיקה ישתנה לפי סוג פרויקט (waterfall מול agile, safety-critical או לא) [L99-L107+20]. **טעות נפוצה:** להשתמש באותה אסטרטגיית בדיקות ללא התייחסות למתודולוגיית הפיתוח (לדוגמה, להכניס את אותן פעילויות סגירה בפרויקט SCRUM קצר – כלומר להחמיץ את מהות האיטרציות הגמישות).

פרק E: האדם הבודק

- **מיומנויות כלליות (Generic tester skills):** בודק טוב ניחן ביסודיות, סקרנות, תשומת לב לפרטים וחשיבה אנליטית [L214-L223+56]. עליו להבין את המערכת (תחום הידע, ידע טכני ובדיקות) ולתקשר בבהירות. כישורים מרכזיים: יכולת תחקיר וניסויים, עבודת צוות, הקשבה פעילה וכתובת דו"חות מדויקים [L214-L223+56]. רמת K2: **זווית מבחן:** לעיתים שואלים לבחון מיהם הכישורים הדרושים (כמו "כישרונות אבחנתיים", "זיהוי באגים קשים", "תיעוד ברור" [L214-L223+56]) **טעות נפוצה:** לזלזל בכישורים חברתיים – בפועל בודקים הם "אלופי חדשות רעות" וצריכים מיומנויות תקשורת גבוהות [L233-L241+56].
- **Confirmation bias ואחרים (הטיות קוגניטיביות):** בודקים נוטים, כמו כל אדם, לעיתים לחפש מידע שמאשר את הציפיות שלהם (לסעות רק גם "אימות קונסטרוקטיבי"), או להחזיק בהנחות עבודה שלא יימתחו [L233-L241+56]. למשל, אם בדיקה אוטומטית מציינת שהכל בסדר, בודק לחוץ עלול לקבל זאת בלי לבדוק הפחתת מקרה. רמת K2: **זווית מבחן:** ייתכן שיביאו תרחיש שבו הבודק נפעל על פי הטייה קודמת (למשל הדגשת הצלחה במקום הסתכלות על מצבי קצה) ושואלים לזהות את ההטייה. **טעות נפוצה:** להתרכז רק בתוצאות שציפינו להן (או רק לפגמים ידועים) ולפספס בעיות חדשות.
- **ביקורת בונה (Constructive feedback):** יש להעביר ממצאי בדיקה באופן ברור ומכבד לפיתוח ולצוותים אחרים. במקום "המוצר שלכם מפספס לולאות ובאגים", יש להציג דוגמאות קונקרטיות, ניתוח תוצאות ולנסח הנחיות לתיקון [L233-L241+56]. רמת K1: **זווית מבחן:** לרוב נבחן ביכולתו של הבודק לשמור על גישה עקרונית לאדם בפיתוח (לא אישית) ולנסח את הדו"ח כשיפור מוצע ולא כהאשמה. **טעות נפוצה:** לתאר באגים כאישיים ("אתם עברתם על הדרישות") במקום להתמקד במערכת ובנתונים.
- **תקשורת בודק-מפתח (Tester-Developer Communication):** קיום תהליך דו-כיווני פתוח וברור בין הבודק למפתח הוא קריטי. המפתח מעוניין בנתונים קונקרטיים על התקלה, והבודק צריך להנגיש זאת (למשל בצילומי מסך או תסריטי שכפולים). רמת K2: **זווית מבחן:** בשאלות על Effective communication יתנו מקרים (למשל מפתח לא מבין דו"ח באגים) ויש לשאול מה היה הדרך המיטבית לדווח אותו. **טעות נפוצה:** מתרגמים דו"ח שגוי (כגון שימוש במונחים עמומים "בעייתיות") במקום להשתמש בביטוי ברור ויעיל, מה שמפחית את האמון.
- **רמות עצמאות הבדיקה (Test Independence):** העצמאות קשורה למי מבצע את הבדיקה: בדיקה עצמית (כאשר היוצר של תוצר מבצע בה את הבדיקה), בדיקה קבוצתית פנימית (עמיתים מאותו צוות), צוות בדיקות פנימי בארגון או צד ג' חיצוני [L285-L293+56]. ככל שהבודק רחוק יותר מכתב הקוד, כך יש פוטנציאל למצוא פגמים שונים [L279-L287+56]. רמת K2: **זווית מבחן:** שאלה טיפוסית תיתן רמת עצמיות (למשל "בדיקה בוצעה על ידי אדם שאינו חלק מצוות הפיתוח") ושואלת מהי רמת העצמאות וכיצד היא משפיעה. **טעות נפוצה:** להסתפק בכך שמפתחים בודקים את הקוד שלהם בלבד, ואז לשבת בציפייה שכל באג יתועד – בפועל, בדיקה עצמאית (לדוגמה בודק ייעודי או ביקורת הצוות) צפויה לחשוף באגים שהמתכנת עצמו פספס בשל הטייה.

- **Whole-team approach (גישת Whole Team):** גישה ממודולוגיית Agile בה כל חברי הצוות (מפתחים, בודקים, אנשי עסק וכו') שותפים לאיכות המוצר [L254-L262†56]. כולם נמצאים במשימה משותפת, משתפים מידע ומשליבים ידע (למשל בודק מכשיר בדיקות קבלה יחד עם אנליסט העסק) [L254-L262†56]. זה משפר שיתוף פעולה ומאפשר שימוש בכישורים שונים לטובת הפרויקט. רמת K1: **זווית מבחן:** בסיטואציה נתונה יש לזהות יתרונות הגישה (תקשורת ושיתוף מידע משופרים, מניעת בורות בין תחומים) [L254-L262†56]. **טעות נפוצה:** לראות באחריות לטעות את האחר (למשל "בודק – זו אשמת המוצר; מפתח – זו אשמת הבודק"), במקום ליישם שיתוף מידע ובדיקה קבועה במהלך הפיתוח.
- **כישורים נוספים (לדוגמה):** יוזכרו בתת-נושאים Generic skills ותקשורת – זוהו. הכישורים הגמישים כוללים גם יכולות טכניות (לדוגמה שימוש בכלי בדיקה), ידע תחומי עמוק בתחום המוצר, יצירתיות ויכולת פתרון בעיות לוגיות [L214-L223†56]. לכל אלה מתלווה רמה נכבדה של מקצועיות ודיוק. רמת K1: K.

טעויות נפוצות של בודקים בנושאי היסודות

- **"אין באגים כי לא נמצאו":** הפרכת העדר שגיאות כמדד לאמינות התוכנה [L121-L124†42] [L20†L64]. בודקים לפעמים משוכנעים שאם הסימולציה הרצתית (או אוטומציה) הצליחה, אז המוצר תקין – אך זה רק מוכיח סולידיות חלקית.
- **המתנה לאחור (Late Testing):** רתיעה מבדיקות מוקדמות (בדיקות בסקירה/ספציפיקציות) מהווה הפרת עיקרון [L20†L76-L83]. Early Testing. כתוצאה, באגים נשארים בלתי נתפסים עד לשלב הקוד המאוחר.
- **ריצה חוזרת של אותם בדיקות (Pesticide Paradox):** שימוש בכפיית אותם תרחישי בדיקה מבלי לעדכן אותם גורם ל"התעייפות בדיקה" – גורם שכאשר מאותו סט בדיקות לא ימצאו עוד באגים [L105-L111†31].
- **ניסיון בבדיקת הכל (Exhaustive testing):** ניסיון לבחון כל שילוב קלטי ותנאי הרצה (לדוגמה, 100% כיסוי נתיב) הוא בלתי מעשי ועלול לגזול זמן רב ולהוריד פוקוס על מהות המערכת [L83-L90†20].
- **העדר מעקב (Traceability):** לא להגדיר מטריצת בדיקה עלול לגרום לכך שדרישות מסוימות "נעדרות" מבדיקות. לדוגמה, אם לא עקבו אחרי תכונה מסוימת לבדיקות, עלולה לצאת גירסה שיצאה עם פונקציונליות ללא בדיקה כלשהי.
- **תקשורת לקויה עם הפיתוח:** מונחים לא ברורים או עזרים לא מספקים (למשל, דו"ח באגים יבש או ללא דוגמאות) עלולים לגרום לאי הבנות (המפתח עלול לבזבז זמן בחיפוש הסבר במקום לפתור את התקלה).
- **Confirmation Bias:** נטייה לוותר או להתעלם מתוצאות סותרות [L233-L241†56]. בודקים וסביבתם עלולים "למצוא רק מה שהם מצפים לו", ולפספס כשלים מפתיעים.
- **שהיה רק בתרחישי "שערויה":** התמקדות ב־happy paths בלבד (למשל בדיקות רק של פלט חיובי כלשהו) מפתיעה כשלים שהיו יכולים להיוותר בגירסאות רמות (לדוגמה, מגרש קוד שלא נבדק כלל).
- **בלבול תפקידים (Testing=QA):** לבלבל בין תחומי האחריות. למשל, לצפות שמבטיח ה-QA יילחץ על כל איפיון ועדיין יבצע בעצמו את כל הבדיקות ידנית (זה טעות, כי QA עוסק בתהליכים ושיטות, בעוד בודק הרצה - רק בבדיקות).
- **ביטחון יתר באוטומציה:** להניח שלא צריך בדיקות ידניות מתאימות כאשר קיימות בדיקות אוטומטיות רבות. דוגמא לטעות: זלזול בפלישות של False Negatives/False Positives במכלול בדיקות אוטומטיות [L97-L105†37] [L125-L133†37].

מבני תרגיל מקובלים

- **זיהוי עיקרון בטעות:** תרחיש המתאר הפרה של עיקרון מסוים (למשל "עמדת QA מפעילה תמיד את אותם מקרי בדיקה ואין לה בדיקות חדשות" – פסטייד פרדוקס) ומבקשים לזהות איזה עיקרון נפרץ.
- **מיון פעילויות לשלבי תהליך:** נתונה רשימה של פעולות (למשל 'כתיבת test plan', 'תיעוד תוצאות בדיקה', 'איסוף דרישות לבדיקות') ויש למיין כל פעילות לשלב המתאים ב-STLC.
- **מיון בין Error/Defect/Failure:** מציגים תרחיש מעשי ומבקשים לסווג האם מדובר בשגיאה אנושית, פגם בקוד או כשל בריצה (לדוגמה, "המתכנת רשם משתנה בשגיאת שם לוגי" = שגיאה; "הקוד מכיל אופרטור שגוי" = פגם; "התוכנה קפצה בשעה שהממשק סופק קלט לא חוקי" = כשל).
- **False Positive/Negative:** למידה בסיטואציות עם תוצאות בדיקה אוטומטיות. לדוגמה, מציגים יומן בדיקה ובודקים האם זוהי תוצאה חיובית שגויה או שלילית שגויה.

- **זיהוי רמת עצמאות:** תרחיש של מי ביצע את הבדיקה (למשל "המחבר של הפונקציה בצע בדיקה") ומתבקש לקבוע רמת עצמאות נמוכה או גבוהה.
- **סינון מלכודות מסיח (פסק זמן):** לתת שאלה מרובת ברירות המכילה תשובות נכונות ו"קריפטיות": יש לזהות את "מלכודת" (למשל תשובה שתקבע משהו מוחלט כמו "תמיד", או צירוף מונחים מבלבל) ולהסביר מדוע זו מלכודת.

סגנון שאלות מבחן אופייני לפרק 1

- **מלכודות עם 'תמיד/אף פעם':** שאלות רבות מציגות תשובות שבהן ניסוחים קיצוניים (כמו "תמיד", "לעולם לא") – אלה כמעט תמיד תשובות שגויות, שכן עקרונות הבדיקה לרוב מנוסחים בגמישות (בדיקה יכולה להקטין סיכון אך אין הבטחה מוחלטת).
- **שאלות מטעות עם מספר מושגים:** שואלים על מושג מסוים (למשל "בדיקת עמוק") אבל התשובות כוללות מושגים קרובים (validation vs verification, error vs failure), ויש ליטול בכך עירנות.
- **פורמט לא סטטי:** בדיקות משמעות לא רק "הרצת קוד". לכן "נכון/לא נכון" בשאלת מהי בדיקה עלול לבדוק אם הבודק מבין שגם סקירות קוד הן בדיקות סטטיות [L207-L215+23].
- **שימוש בריבוי ברירות:** בדוגמאות בחינה נפוצים תרחישים המשלבים שני עקרונות או יותר, כאשר התשובה האחת מכילה "שתי תכונות נכונות" והאחרות מציגות רק חצי אמת.
- **שאלות עם דגש על ניתוח:** במבחן ISTQB בחירת תשובה נכונה לרוב דורשת שינתח את המצב ויבחר באופציה שמבטאת את הרעיון המדויק (למשל "מה הטענה הנכונה לגבי הבדיקה?").

הקשר ישראלי – ראיון עבודה ומונחים

- **דוגמאות בשאלות ראיון בארץ:** בבדיקות ידניות בראיונות בישראל נפוצות שאלות כמו "מה ההבדל בין בדיקות Smoke ל-Sanity", "מה הסיבה לקיום בדיקות רגרסיה", "מהו life cycle של הבדיקה". פעמים רבות נשאלים גם על העקרונות הבסיסיים: למשל "אם לא מצאת באגים, מה זה אומר?" (בדיקה של עיקרון ה-absence-of-errors) [L64-L68+20], או "האם בודק טוב הוא מי שמצא הכי הרבה באגים? למה לא?" (נושא מונחה על איכות כללית).
- **מונחים בעברית/אנגלית נפוצים:** בראיונות משתמשים במונחים בשתי השפות. למשל: שגיאה (error) או mistake, ייתכן פשוט "טעות", פגם/באג (defect/bug), כשל (failure). הבדיקה ידועה בשם testing (לעתים מחליפים QA – אך זה שגוי, ראו לעיל), תוכנית בדיקות = Test Plan, תרחיש בדיקה = Test Case, מסמך דרישות = Requirements Document. Traceability matrix מכונה גם מטריצת מעקב. גם "בדיקות רגולריות" (Regression) ו"בדיקות דוחפניות" (Smoke/Sanity) מוכרות ככאלה, וכן terms כמו coverage (כיסוי) ו-monitoring (ניטור). השאלה על העצמאות עשויה להיות "מי עשה את הבדיקה – מפתח, בודק פנימי או חיצוני" ולצפות להבחנה ברמות העצמאות הנ"ל (למשל "בדיקה עצמית" מול "בודק ייעודי").
- **הקשר בין מושגים ושפות:** יש לשים לב כי לפעמים מציגים ערכים בשני השפות: למשל בפועל "פגם" עשוי להיקרא גם "bug" בצ'ט סכני, ו"בודק תוכנה" לעיתים מתפרש בראיון כ-QA engineer. חשוב להבהיר בהקשר ולא לערבב מושגים (למשל להבין ש-Quality Assurance הוא תחום מניית התהליכים, לא שם לתפקיד לבדיקה).

מקורות

מקור (URL)	מו"ל	שנה	רישיון
Syllabus CTFL v4.0.1 [1+L489 L493]	ISTQB (International Software Testing Qualifications Board)	2024	International © Software Testing Qualifications Board
What is Testing? (ASTQB) 1.1 [L245-L253+23] [L281]	ASTQB (American Software Testing Qualifications Board)	–	unclear
Testing vs QA vs QC (Testsigma) [27+L54-L61] [L46-L54+27]	Testsigma (Paperblog)	2020	unclear

מקור (URL)	מחבר	שנה	רישיון
Principles (CodeGym) 7 -20†L99】 【L83-L90†20】 【L107	CodeGym University	2023	unclear
Principles (MoT forum) -31†L105】 【L81-L85†31】 【L111	Ministry of Testing (The Club	2019	unclear
Error, Defect, Failure (ToolsQA) -34†L201】 【L143-L147†34】 L205】 【34†L268-L270】	ToolsQA.com	-2013 2026	2013-2026 © ToolsQA.com (All rights reserved)
False Pos/Neg (Testfully) -37†L125】 【L97-L105†37】 【L133	Testfully.io	2021 .upd) (2024	unclear
STLC Phases (FulcrumRocks) -48†L139】 【L70-L78†48】 【L146	FulcrumRocks	2022	Copyright 2022 © FulcrumRocks
-Test Planning (TryQA) 【45†L71 L79】 【45†L103-L112】	TryQA blog	-	unclear
-ISTQB Cheat Sheet 【42†L121 L124】 【42†L158-L161】	ISTQB.guru	2026	unclear
סקירת מושגים בדיקות (HackerU) 【L50-L58†12】	HackerU blog	2026	unclear